

7. Kanban

7. Kanban

Nella sezione precedente hai visto Scrum: sprint a tempo fisso, ruoli definiti, eventi ricorrenti (Sprint Planning, Daily Scrum, Sprint Review, Sprint Retrospective). Immagina il team di **ShopFacile** appena uscito da una Sprint Retrospective: tra i punti emersi c'è che i bug urgenti segnalati dagli utenti, arrivati a metà sprint senza preavviso, hanno continuato a “saltare la fila” e a scombinate la pianificazione fatta a inizio sprint. Sara e Luca si chiedono se, almeno per questo tipo di lavoro imprevedibile, un flusso continuo non sia più adatto di uno sprint a tempo fisso. È esattamente il problema che Kanban è pensato per risolvere.

Scrum è un framework molto “strutturato”. Kanban parte da un'idea diversa e più semplice: **rendere visibile il lavoro e limitare quanto ne facciamo contemporaneamente**, senza necessariamente imporre sprint o ruoli fissi.

Se Scrum è come organizzare il lavoro a “capitoli” di durata fissa, Kanban è più simile a un nastro trasportatore continuo: il lavoro scorre, una voce alla volta, dall'inizio alla fine, senza tappe temporali predefinite.

Obiettivi della sezione

Alla fine di questa sezione saprai:

- da dove viene il metodo Kanban e perché si chiama così;
- come è strutturata una board Kanban e cosa rappresentano colonne e card;
- cos'è il WIP (Work In Progress) e perché limitarlo migliora il flusso di lavoro del team;
- la differenza tra Lead Time e Cycle Time, con un esempio numerico;
- quando conviene usare Kanban invece di Scrum (o entrambi insieme).

7.1 Da dove viene Kanban

La parola **Kanban** (看板) viene dal giapponese e significa più o meno “cartellino visivo” o “insegna”. Il metodo nasce **non nel software**, ma in fabbrica: negli anni '40-'50 **Toyota** lo introdusse nelle sue linee di produzione automobilistica come parte del sistema di produzione “lean” (cioè “snello”, senza sprechi).

L'idea originale era molto concreta: in una linea di montaggio, ogni reparto usava dei cartellini fisici per segnalare al reparto precedente “ho bisogno di altri pezzi” — invece di produrre pezzi in grandi quantità “a caso” e ammassarli in magazzino (con il rischio di produrne troppi o troppo pochi), si produceva **solo quello che serviva, quando serviva**. Questo riduceva sprechi, magazzini pieni di materiale inutilizzato, e colli di bottiglia nascosti.

Decenni dopo, a partire dagli anni 2000, questa filosofia è stata adattata al lavoro di team software (il nome più associato a questa trasposizione è David J. Anderson). L'idea di fondo resta identica: **rendere visibile il flusso di lavoro e non accumulare più lavoro “in corso” di quanto il team riesca effettivamente a gestire.**

7.2 La board Kanban: colonne e card

Il cuore pratico di Kanban è la **board** (bacheca), divisa in **colonne** che rappresentano le fasi che un elemento di lavoro attraversa, da quando viene richiesto a quando è completato.

La versione più semplice ha tre colonne:

- **To Do** (da fare): lavoro non ancora iniziato;
- **In Progress** (in corso): lavoro su cui qualcuno sta lavorando ora;
- **Done** (fatto): lavoro completato.

Nella pratica reale, quasi nessun team si ferma a tre colonne: le colonne vengono **personalizzate** per rispecchiare le fasi reali del lavoro del team. Un esempio molto comune in un contesto software:

To Do → In Analisi → In Sviluppo → In Test → Done

Ogni **card** (cartellino, che riprende proprio l'idea del cartellino giapponese originale) rappresenta un singolo elemento di lavoro: una user story, un bug da correggere, un task tecnico. La card si sposta da sinistra a destra sulla board, colonna dopo colonna, man mano che avanza. Ogni card contiene tipicamente: un titolo, una breve descrizione, chi ci sta lavorando (assegnatario), e a volte etichette o una stima di dimensione.



Analogia: pensa alla board come al monitor delle partenze in un aeroporto. Ogni volo (card) parte da “In attesa”, passa per “Imbarco in corso” e arriva a “Partito”. A colpo d'occhio, chiunque guardi il monitor capisce subito lo stato di ogni volo — senza dover chiedere a nessuno “a che punto siamo?”.

Le board Kanban possono essere fisiche (un muro con post-it) o digitali (strumenti come GitHub Projects, Trello, Jira). Su GitHub, ad esempio, una board di questo tipo si costruisce con **GitHub Projects**: le colonne diventano stati personalizzabili e le card possono collegarsi direttamente a Issue e Pull Request del repository, così lo stato del lavoro sul codice si riflette automaticamente sulla board.

Esempio pratico: immagina la board Kanban del team di **ShopFacile**, con 4 colonne — **To Do**, **In Sviluppo**, **In Test**, **Done** — e in un dato lunedì mattina la situazione è questa:

- **To Do** (6 card): richieste appena arrivate, non ancora prese in carico — es. “Aggiungere validazione campo email”, “Bug: pagina lenta su mobile”.
- **In Sviluppo** (3 card): “Fix errore export PDF” (assegnata a Marco), “Aggiornare libreria di logging” (assegnata a Giulia), “Aggiungere filtro categoria al catalogo” (assegnata ad Ahmed).
- **In Test** (1 card): “Nuovo filtro ricerca prodotti”, in verifica dal tester.
- **Done** (3 card, completate questa settimana): “Fix bottone disabilitato”, “Aggiornamento pagina contatti”, “Correzione traduzione IT”.

Guardando solo questa riga di numeri (6 / 3 / 1 / 3), un Project Manager può già farsi una prima idea del flusso: c'è più lavoro in attesa (6) di quanto il team ne stia lavorando attivamente (3+1=4) — non è necessariamente un problema, ma è un segnale da tenere d'occhio se il numero in “To Do” continua solo a crescere.

Avere una board così, visibile a tutti, è già un grande passo avanti — ma non basta da solo: bisogna anche controllare **quanto lavoro il team si mette a fare contemporaneamente**, non solo renderlo visibile. È qui che entra in gioco il concetto di WIP.

7.3 WIP: il lavoro “in corso” e i suoi limiti

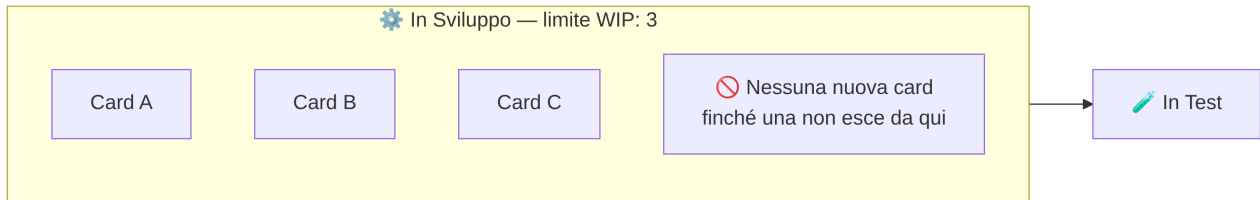
WIP è l'acronimo di **Work In Progress**, cioè “lavoro in corso”: il numero di elementi che, in un dato momento, sono già iniziati ma non ancora completati.

Uno dei principi cardine di Kanban è il **limite di WIP**: si stabilisce un numero massimo di card che possono trovarsi contemporaneamente in una determinata colonna (tipicamente le colonne di lavoro attivo, come “In Sviluppo” o “In Test”). Se una colonna ha già raggiunto il suo limite, **nessuno può iniziare una nuova card in quella colonna** finché non se ne libera una completandola e facendola avanzare.

Detto così può sembrare controintuitivo: perché limitare volontariamente quanto lavoro il team può iniziare? La risposta è che **fare tante cose contemporaneamente, in realtà, le fa fare tutte più lentamente**.

Analogia: pensa al casello di un'autostrada durante un weekend di esodo. Se tutte le macchine potessero entrare in autostrada contemporaneamente senza controllo, si formerebbe un ingorgo che blocca *tutti*, e alla fine le macchine arriverebbero più tardi di quanto arriverebbero se il casello facesse passare le auto in modo controllato, poche alla volta. È lo stesso principio di un **imbuto**: se versi il liquido troppo in fretta, l'imbuto si intasa e in realtà il liquido scende **più lentamente** rispetto a versarlo con un flusso costante e misurato.

Nel lavoro di un team succede la stessa cosa: se ognuno ha 6 task “aperti” contemporaneamente, continua a passare da uno all'altro (context switching), nessun task avanza davvero in fretta, e la sensazione di “essere sempre indaffarati” nasconde il fatto che **poco viene realmente completato**. Limitare il WIP forza il team a **finire prima di iniziare altro**, il che in pratica fa terminare più lavoro, più velocemente, con meno errori (perché ci si concentra su meno cose alla volta).



Un segnale tipico da Project Manager/Scrum Master: se vedi una colonna della board costantemente “esplosa” oltre il suo limite di WIP, è un sintomo concreto di un **collo di bottiglia** nel flusso di lavoro del team, su cui vale la pena indagare (manca una competenza? una fase richiede un’approvazione lenta? c’è troppa dipendenza da una sola persona?).

Esempio pratico: nel team di ShopFacile, la colonna “In Sviluppo” ha un limite di WIP fissato a 3. Martedì la colonna ha già 3 card (A, B, C) e **Ahmed**, che ha appena finito un task, vorrebbe iniziarne uno nuovo, la card D. Con il limite di WIP attivo, **non può farlo**: deve prima aiutare a completare A, B o C (magari facendo code review a un collega, o testando manualmente una delle tre) e farla avanzare in “In Test”. Solo quando una delle tre card lascia la colonna, si “libera uno slot” e la card D può entrare. Il risultato pratico è che Ahmed, invece di aprire un quarto fronte, spinge a chiudere qualcosa che è già a metà — ed è esattamente l’effetto voluto.

Limitare il WIP non serve solo a evitare che il team apra troppi fronti contemporaneamente: come vedremo subito, avere poche card “in corso” alla volta rende anche possibile **misurare con precisione** quanto tempo impiega davvero una card ad attraversare la board, dall’ingresso in To Do al completamento.

C’è però un effetto collaterale del limite di WIP che nessun manuale racconta volentieri: quando la colonna è piena e qualcuno resta senza un task “proprio” da avviare, si sente in colpa, come se stesse “non facendo niente”. È un riflesso comprensibile — siamo abituati a misurare il proprio valore dal numero di cose che abbiamo aperto — ma è esattamente il comportamento opposto a quello che il limite di WIP vuole ottenere. Il compito di chi facilita il team (che tu incontrerai nella pratica come Scrum Master, vedi la sezione 6.2.2) è ricordare, con costanza, che **aiutare a chiudere il lavoro di un collega è lavoro**, non meno prezioso di aprirne uno nuovo: è proprio quello che fa Ahmed nell’esempio sopra, e non è “tempo perso” solo perché non produce una nuova card con il suo nome sopra.

Classi di servizio e corsia di emergenza: e i bug urgenti?

Il capitolo si è aperto con la domanda di Sara e Luca: cosa si fa quando arriva un bug critico che non può aspettare il proprio turno in coda? La risposta di Kanban non è “si ignora la board e si lavora fuori sistema”: è definire, **prima** che il problema si presenti e insieme al team, una **classe di servizio** dedicata all’urgenza reale, spesso visualizzata come una corsia separata in alto sulla board (in inglese si parla di *swimlane*).

Tre decisioni vanno prese in anticipo, non nel momento della crisi:

- **Quante corsie di emergenza esistono.** Tipicamente **una sola**: se ogni tipo di richiesta ha la sua corsia “prioritaria”, nessuna lo è davvero più delle altre, e il limite di WIP perde significato.
- **Quali criteri rendono un lavoro davvero urgente.** Non basta “il cliente dice che è urgente”: un criterio concreto per ShopFacile potrebbe essere “il sito è irraggiungibile o un cliente non riesce a pagare, in produzione, in questo momento” — non “sarebbe comodo averlo prima”.
- **Cosa succede al lavoro normale quando la corsia di emergenza viene usata.** La card urgente entra e viene lavorata subito, anche superando il limite di WIP della colonna — ma il team ne discute apertamente nel Daily successivo, perché quel superamento ha un costo: qualcos'altro, inevitabilmente, avanza più lentamente.

Esempio pratico: tornando al team di ShopFacile, dopo la Retrospective con cui si è aperto questo capitolo, Sara e Luca definiscono insieme al team una singola corsia di emergenza, riservata solo a bug che bloccano un pagamento o rendono il sito irraggiungibile. La settimana dopo arriva esattamente un caso così: un bug impedisce a una parte dei clienti di completare il checkout. La card entra nella corsia di emergenza, **Giulia** la prende in carico subito, superando per un giorno il limite di WIP della colonna “In Sviluppo”. Nel Daily seguente il team ne discute apertamente: la card “Aggiornare libreria di logging” che stava a metà è rimasta ferma un giorno in più, ma è una conseguenza accettata, non un imprevisto subito in silenzio. I bug urgenti non “saltano la fila” in modo caotico e invisibile come prima: hanno una fila propria, dichiarata e con regole chiare.

7.4 Lead Time e Cycle Time

Kanban, essendo un metodo orientato al “flusso” continuo di lavoro, si misura soprattutto con due metriche di tempo, spesso confuse tra loro ma concettualmente diverse. Vediamole applicate alla board di ShopFacile che abbiamo appena descritto.

Lead Time

Il **Lead Time** è il tempo totale che passa **dal momento in cui una richiesta viene fatta** (la card entra nella colonna “To Do”, cioè nel backlog) **al momento in cui viene completata** (la card arriva in “Done”). È il tempo che interessa di più a **chi ha fatto la richiesta** (un cliente, uno stakeholder): “quanto tempo devo aspettare per avere questa cosa?”.

Cycle Time

Il **Cycle Time** è il tempo che passa **dal momento in cui il team inizia effettivamente a lavorarci** (la card entra in “In Progress” o “In Analisi”) **al momento in cui viene completata**. È il tempo che interessa di più al **team**, perché misura quanto efficiente è il lavoro una volta avviato, senza contare l'attesa in coda.

La differenza pratica è quindi il tempo di **attesa in coda**, prima che qualcuno inizi effettivamente a lavorare sulla richiesta.

Esempio numerico

Immagina questa card: “Aggiungere filtro di ricerca alla pagina prodotti”.

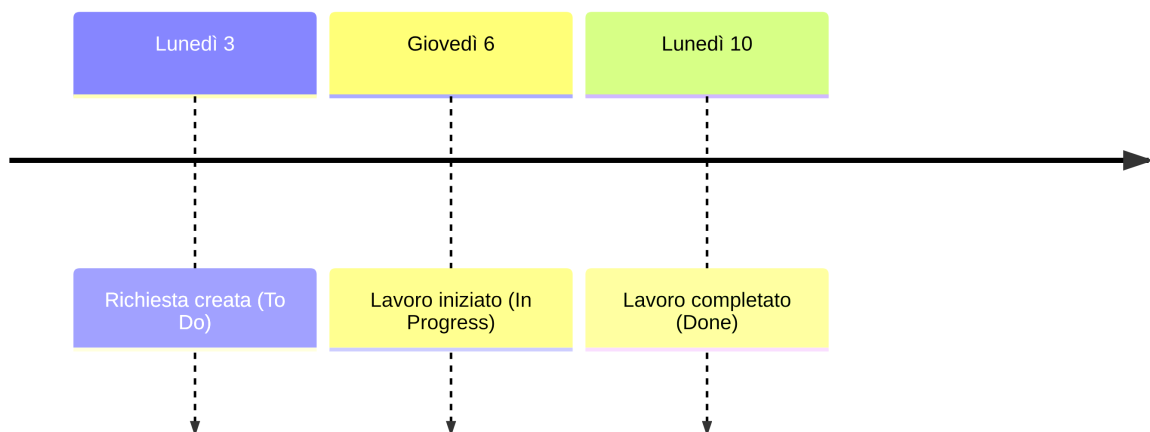
- **Lunedì 3**: la richiesta viene registrata e messa in “To Do”.
- **Giovedì 6**: uno sviluppatore inizia effettivamente a lavorarci (“In Progress”).
- **Lunedì 10**: il lavoro è completato e la card arriva in “Done”.

Calcoliamo:

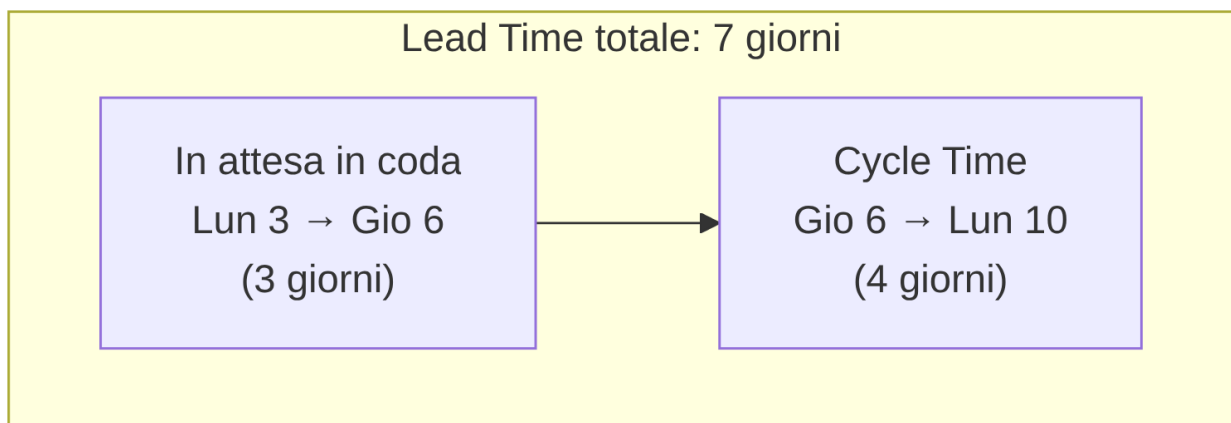
- **Lead Time** = da lunedì 3 a lunedì 10 = **7 giorni**
- **Cycle Time** = da giovedì 6 a lunedì 10 = **4 giorni**

La differenza (3 giorni) è il tempo che la richiesta ha passato “in coda”, in attesa che qualcuno se ne occupasse.

Lead Time vs Cycle Time



Rappresentazione a barre, per confrontare visivamente i due intervalli:



Perché la distinzione conta per un Project Manager: se il cliente si lamenta che “le richieste ci metton troppo ad arrivare”, devi capire **dove** si perde tempo. Se il Cycle Time è basso ma il Lead Time è alto, il problema non è che il team lavora lentamente: è che le richieste restano troppo tempo in coda prima che qualcuno le prenda in carico. Sono due problemi diversi, con soluzioni diverse (il primo si risolve migliorando l’efficienza del team, il secondo migliorando come si priorizza e si smista il lavoro in arrivo).

Esempio pratico: confrontiamo due card diverse della stessa settimana.

- Card “Bug critico: pagamento non va a buon fine” — entra in To Do **martedì 4** alle 9:00, un developer la prende in carico **subito** (martedì 4, 9:30) perché è urgente, ed è completata **mercoledì 5**. Lead Time $\approx$ Cycle Time $\approx$ **1 giorno**: nessuna attesa in coda.
- Card “Migliorare testo pagina ‘Chi siamo’” — entra in To Do **martedì 4**, ma essendo bassa priorità resta in coda per **12 giorni**; qualcuno la prende in carico **giovedì 16** e la completa lo stesso giorno in 2 ore. Lead Time = **12 giorni**, Cycle Time = **meno di 1 giorno**.

Stesso team, stessa efficienza di esecuzione (Cycle Time basso in entrambi i casi) — ma un Lead Time completamente diverso, perché dipende da **quanto la card ha aspettato in coda**, non da quanto ci ha messo il team a farla una volta iniziata.

Lead Time e Cycle Time sono le metriche con cui Kanban misura il flusso continuo di lavoro; Scrum, che lavora a sprint, misura invece l’avanzamento con una metrica diversa, la Velocity. Vale la pena confrontare i due approcci fianco a fianco, per capire quando ha senso usare l’uno o l’altro.

7.5 Kanban vs Scrum: quando usare cosa

Kanban e Scrum condividono lo stesso spirito Agile (visto nella sezione 5), ma organizzano il lavoro in modo diverso.

	Scrum	Kanban
Struttura del tempo	A sprint di durata fissa (es. 2 settimane)	Flusso continuo , senza intervalli di tempo fissi
Consegna del lavoro	Un “pacchetto” di elementi consegnato a fine sprint	Ogni elemento viene consegnato appena è pronto
Ruoli	Ruoli definiti (Scrum Master, Product Owner, Team)	Nessun ruolo obbligatorio specifico
Eventi ricorrenti	Planning, Daily, Review, Retrospettiva	Nessun evento obbligatorio (anche se molti team fanno comunque una daily)
Cosa si limita	La quantità di lavoro pianificata per lo sprint (lo Sprint Backlog)	Il WIP, cioè quante card possono essere “in corso” in ogni momento
Metriche tipiche	Velocity (punti completati per sprint)	Lead Time, Cycle Time, Throughput
Cambiamenti a metà lavoro	Scoraggiati a metà sprint (si aspetta il prossimo)	Benvenuti in qualsiasi momento, il flusso è continuo

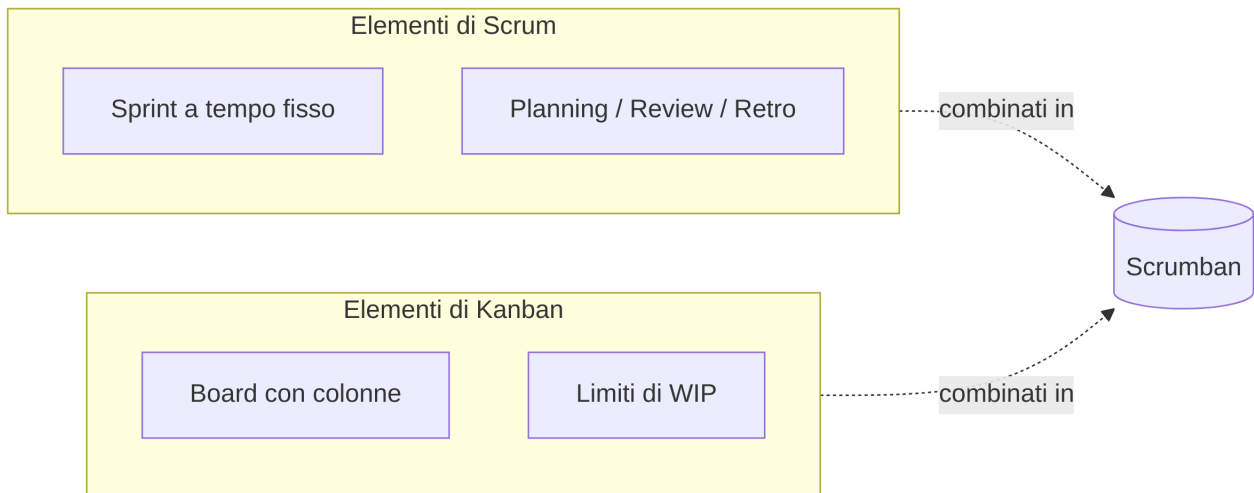
Nella tabella compare un termine che non hai ancora incontrato: il **Throughput** (letteralmente “capacità di elaborazione”) è semplicemente **quante card il team completa in un dato periodo** (es. “12 card a settimana”), a differenza di Lead Time e Cycle Time che misurano una durata. Se Lead Time e Cycle Time rispondono a “quanto tempo ci vuole”, il Throughput risponde a “quanto ne finiamo” — le due informazioni insieme danno un quadro completo del flusso: un team può avere un Cycle Time basso ma un Throughput scarso, se lavora bene sulle poche card che porta avanti ma ne avvia troppo poche in parallelo.

Quando usare Scrum: quando il lavoro si presta a essere pianificato in blocchi (sprint), il team vuole momenti regolari di revisione e retrospettiva, e serve una cadenza predicibile per pianificare e comunicare con gli stakeholder (es. “a fine sprint mostriamo una demo”).

Quando usare Kanban: quando il lavoro arriva in modo **imprevedibile e continuo**, con priorità che cambiano spesso — è il caso tipico di team di **supporto, manutenzione, gestione incident** o di piattaforme DevOps, dove non ha molto senso “pianificare due settimane fisse” perché una richiesta urgente (es. un bug critico in produzione) può arrivare in qualsiasi momento e deve poter “saltare la fila”.

Scrumban: il meglio dei due mondi

Molti team reali non scelgono in modo rigido l'uno o l'altro, ma **combinano i due approcci**, in un ibrido informalmente chiamato **Scrumban**: si mantiene la cadenza degli sprint e degli eventi Scrum (planning, review, retrospettiva) per la pianificazione e la comunicazione, ma si gestisce il lavoro **dentro** lo sprint con una board Kanban e limiti di WIP, per governare meglio il flusso quotidiano. È un approccio molto comune nei team che gestiscono sia nuovo sviluppo pianificato sia richieste impreviste (bug urgenti, richieste di supporto).



Prima di chiudere, però, un'ultima onestà intellettuale: Kanban non è privo di costi. La sua stessa forza — l'assenza di una cadenza fissa — è anche il suo limite principale. Senza sprint, è più difficile dare al cliente una data di consegna prevedibile per un gruppo di funzionalità ("quando avremo tutte queste 5 cose insieme?") ha una risposta più naturale in Scrum che in Kanban, dove ogni card procede per conto proprio). E senza gli eventi ricorrenti di Scrum, mancano i momenti naturali in cui il team si ferma a fare una demo o a riflettere su come sta lavorando: una Review e una Retrospective non nascono da sole nel flusso continuo, vanno **istituite deliberatamente** — molti team Kanban le mantengono comunque, a cadenza fissa, anche senza uno sprint che le "obblighi".

Chiudiamo qui il confronto con Scrum: riprendiamo in breve i punti chiave di questa sezione su Kanban.

7.6 Riepilogo

- Kanban nasce nella produzione industriale (Toyota, filosofia "lean") e si è poi adattato al lavoro dei team software.
- La **board**, con le sue colonne personalizzabili, rende visibile lo stato di ogni card.
- Limitare il **WIP** (lavoro in corso) migliora il flusso complessivo: meno cose iniziate contemporaneamente, più cose finite rapidamente.
- **Lead Time** misura l'attesa dal punto di vista di chi richiede; **Cycle Time** misura l'efficienza del team una volta che il lavoro è iniziato.
- Scrum lavora a **sprint**, Kanban lavora a **flusso continuo**: la scelta dipende dal tipo di lavoro del team, e i due approcci si possono anche combinare (**Scrumban**).

Esercizi pratici

1. **Disegna una board a 4 colonne.** Su carta o con un tool gratuito online, crea una board con le colonne **To Do** → **In Sviluppo** → **In Test** → **Done** e inventa 6 card realistiche per un progetto a tua scelta (anche immaginario, es. un sito di prenotazioni). Distribuisci le card nelle colonne

come faresti se fossi tu a gestire il flusso in questo momento. **Come verificare:** se riesci a spiegare a voce, per ciascuna card, “perché è in quella colonna e non in un'altra”, l'esercizio è fatto bene.

2. **Applica un limite di WIP e osserva l'effetto.** Riprendi la board dell'esercizio 1 e imposta un limite di WIP di 2 sulla colonna “In Sviluppo”. Se hai messo più di 2 card in quella colonna, decidi quali spostare (indietro in To Do, o avanti se sono davvero pronte) per rispettare il limite.
Come verificare: alla fine, la colonna “In Sviluppo” deve contenere esattamente 2 card (o meno), e devi saper indicare quale card verrebbe “sbloccata” per prima quando una delle due card attuali viene completata.
3. **Calcola Lead Time e Cycle Time con date a tua scelta.** Inventi una card con tre date: quando entra in “To Do”, quando qualcuno la prende in carico (“In Sviluppo”), quando viene completata (“Done”). Calcola a mano Lead Time e Cycle Time, ed evidenzia quanti giorni sono di “attesa in coda”. **Come verificare:** Lead Time deve sempre essere **maggiore o uguale** al Cycle Time (mai il contrario) — se il tuo calcolo dà un Cycle Time più alto del Lead Time, hai invertito una data.
4. **Osserva la board reale del team.** Chiedi a un collega di farti vedere la board Kanban (o Scrum) usata realmente nel progetto: annota quante colonne ha, se ci sono limiti di WIP visibili e su quali colonne, e quante card ci sono in ciascuna colonna in questo momento. **Come verificare:** sei in grado di riportare, senza guardare di nuovo la board, il nome esatto di tutte le colonne e almeno un esempio di card per ciascuna.
5. **Individua un possibile collo di bottiglia.** Guardando la stessa board reale (o quella dell'esercizio 1), individua se c'è una colonna con più card delle altre, e formula un'ipotesi sul perché (manca una competenza? un'approvazione lenta? una sola persona sovraccarica?).
Come verificare: prova la tua ipotesi con un collega o con la tua collega Scrum Master/PM — se conferma (anche parzialmente) la tua lettura, hai capito il concetto.
6. **Confronta Scrum e Kanban su un caso reale.** Pensa a un tipo di richiesta che arriva spesso nel progetto (es. un bug urgente in produzione, oppure una nuova feature pianificata) e scrivi 2-3 righe su quale dei due approcci (Scrum a sprint, o Kanban a flusso continuo) si adatterebbe meglio a gestirla, e perché. **Come verificare:** la tua risposta deve citare almeno uno dei criteri visti nella tabella della sezione 7.5 (es. prevedibilità del lavoro, necessità di “saltare la fila”, cadenza di consegna).

Collegamenti

- [4. Git e GitHub](#) — dove Issue e Pull Request diventano le card di una board Kanban costruita con GitHub Projects
- [8. Project Management](#) — come usare le metriche di flusso (Lead Time, Cycle Time, Throughput) per monitorare un progetto

Risorse

- [Kanbanize — What is Kanban?](#)
- [Atlassian — Kanban](#)
- [Atlassian — Lead Time vs Cycle Time](#)
- [Atlassian — What is WIP limit?](#)
- [Kanban University — What is Kanban?](#)
- [Scrumban — Definizione ed esempi](#)